

[illegible]

This exam consists of 17 questions.

Points are indicated for each question and broken down to each part.

Question 1: Software Engineering (3 points)

Ordnen Sie Software Engineering auf einer Skala von Handwerk bis Ingenieurwissenschaft ein. Geben Sie zwei Argumente für Ihre Zuordnung an.

Map Software Engineering on a scale from craft to engineering. Give two arguments for your mapping.

Marking:

1 mark for "somewhere in between"

See Slide L01, S. 22

1 mark each for each argument (0 marks if direction towards craft/engineering is not clear; only .5 marks if production/science are mentioned but not specific to software engineering)

Question 2: Software Development Phases (5 points)

Nennen Sie die traditionellen Phasen der Softwareentwicklung. Geben Sie konkrete Beispielaktivitäten an, um jede Phase bei der Entwicklung einer Chatbot-App zu veranschaulichen.

List the traditional software engineering phases. Provide concrete example activities to illustrate each phase while developing a chatbot app.

Marking:

2.5 marks for 5 traditional phases: Requirements engineering, Design, Implementation, Verification and validation, Maintenance (a.k.a. evolution) -0.5 for each missing/incorrect phase

2.5 marks for each of activity, 0 marks if activity is not related or specific to chatbot

Question 3: SCRUM vs. Waterfall Model (6 points)

Wählen Sie drei Aspekte von SCRUM aus, stellen Sie diese dem Wasserfall-Modell gegenüber und erklären Sie für jeden Aspekt einen von SCRUM beabsichtigten Vorteil.

Pick three aspects of SCRUM, contrast them with the Waterfall model, and explain the intended benefit of each aspect of SCRUM.

Marking:

1 mark x 3 for mentioning and matching to corresponding aspect of waterfall model (only 0.5 if aspect is not matched to Waterfall model)

- Several rounds of development: system concept, requirements, design
- In each round, mitigate risks: Define objectives of part you are doing, Map alternatives for implementation, Recognize constraints on these alternatives, Use prototyping, analysis, etc. to gain necessary knowledge and reduce risk, Plan the next step
- At the end, perform sequence of coding, testing, and integration

1 mark for reasonable benefit

must come in triplets; (spiral, waterfall, benefit), not enough to list one benefit or general ones that do not match the contrasted elements

Question 4: Software Engineering Mistakes (6 points)

Nennen Sie 3 typische Fehler in der Softwareentwicklung aus der Vorlesung und nennen Sie für jeden ein Vorgehensmodell, welches diesen Fehler am besten adressiert. Begründen Sie ihre Antworten knapp.

List 3 classic software engineering mistakes from the lecture and name for each one process model that best addresses it. Briefly justify your answer.

Marking:

1 mark x 3 for each mistake:

- scheduling: short phases with identifying highest risks
- planning issues: see above
- gold plating requirements: many iterations and evaluation, force progress to next issue
- silver bullet: evaluation in every iteration

1 mark x 3 for the mitigation of each mistake. Mitigation has to be specific to each point (this is not a recall, but a transfer task)

Question 5: Stakeholders (4 points)

Nennen Sie vier Stakeholder für eine Chatbot App und geben Sie für jeden ein relevantes Interesse (spezifisch für den Chatbot) für die Software an.

Name four stakeholders for the software of a chatbot app and provide a relevant concern (specific to the chatbot) for the software for each.

Marking:

1 mark for every stakeholder and relevant concern

if no concerns are given, at most one mark in total

- Developers need specification and have to develop software
- ...

Question 6: Requirements classification (4 points)

Diskutieren Sie für die folgenden Anforderung, ob diese funktional oder nicht-funktional sind. Begründen Sie dies in je einem Satz.

For each of the following requirements, discuss whether they are functional or non-functional. Justify each answer in one sentence.

- (a) *Eltern haben Zugriff auf die Chatverläufe ihrer Kinder.*
Parents have access to their children's chat histories.

Marking:

functional: clearly a function to provide

- (b) *Der Chatbot sollte niemals Aktivitäten vorschlagen, die den Benutzer gefährden.*
The chatbot should never suggest activities that put the user at risk.

Marking:

could be both, depends whether the focus is on the interpretation of *putting at risk* or the functionality of filtering it

Question 7: User stories (4 points)

Schreiben Sie zwei User Stories als Anforderungen an eine Chatbot App.

Write two user stories as requirements for a chat bot app.

Marking:

2 marks for each, where -1 mark for missing any of the three elements: role, capability, value

Example: "As a parent, I want to inspect my childrens' chat histories in order to make sure they are not harmed."

Example: "As a user, I want the robot to suggest diverse ideas in order to inspire me to try new things."

0 if it is not a user story

Question 8: Class Diagram vs. Object Diagram (4 points)

Nennen Sie eine Gemeinsamkeit und einen Unterschied zwischen Klassendiagrammen und Objektdiagrammen. Geben Sie ein Anwendungsbeispiel an, in dem ein Diagrammtyp passender ist als der andere.

List a commonality and a difference between Class Diagrams and Object Diagrams. Give an example task where one diagram type would be used over the other.

Marking:

1 mark for commonality, e.g., model structure rather than behavior (not really that they contain classes as this is only for CDs and ODs contain class names to indicate types, but that is quite a different use)

1 mark for difference, e.g., concrete example vs. all possible structures

2 marks for the example: 1 mark for relevance, 1 mark for correctness of choice of diagram

Question 9: Incompleteness (4 points)

Schlagen Sie eine unvollständige Anforderung vor und geben Sie eine Variante der Anforderung mit höherer Vollständigkeit an. Begründen Sie anhand eines knappen Szenarios, warum hier Unvollständigkeit wünschenswert sein könnte.

Provide an incomplete requirement and provide a variant of the same requirement that is more complete. Argue why incompleteness might be desired or necessary for your example requirement by providing a brief scenario.

Marking:

1 mark for the incomplete (some cornercases are missing), 1 mark for the more complete requirement only one in total, if the two requirements are not related

1 mark for example scenario and 1 mark for argument (some cases might be unknown or aer better resolved by developers later)

Question 10: Model Criteria (6 points)

Nennen und erläutern Sie (jeweils maximal einen Satz) die drei Modellkriterien der Modelldefinition. Veranschaulichen Sie jedes Kriterium anhand eines Feature Models.

List and explain (max. one sentence each) the three criteria of the definition of models. Illustrate them on a feature model.

Marking:

1 mark per element and 1 mark per explanation

- references an original
- abstracts the original
- is usable for a specific purpose

only 1 mark in total if explanations are missing

Question 11: Implementation guidelines (6 points)

Nennen Sie drei Richtlinien zum Benennen von Methoden, Variablen oder Klassen und begründen Sie jede davon knapp.

List three guidelines for naming methods, variables, or classes, and for each, provide a brief rationale.

Marking:

1 mark for each guideline, e.g., short only if scope is short, caps and camel case, use of active words for methods etc.

1 mark for a rationale, e.g., reduce typing effort but larger scope required more uniqueness and meaningful names, distinguish class names from var names, package names, and method name, etc.

Question 12: Testing techniques (4 points)

Nennen Sie zwei Techniken für White-Box-Tests und beschreiben Sie jeweils einen Vorteil.

Name two white-box testing techniques and give one advantage of each.

Marking:

1 mark for each technique, e.g., coverage testing, mutation testing, (inspection as a stretch)

1 mark for benefits each (benefits must be explicitly mentioned as such, even if part of the definition is repeated)

Question 13: Combinatorial test design (8 points)

Verwenden Sie den In-Parameter-Order-Algorithmus, um den gegebenen Testplan für eine paarweise Abdeckung ($t = 2$) zu vervollständigen.

Use the In-Parameter-Order algorithm to complete the test plan below for pairwise coverage ($t = 2$).

Parameter: $Player = \{white, black\}$, $AI = \{ruleBased, alphaBeta\}$, and $Timed = \{on, off\}$.

Testplan:

<i>white</i>	<i>ruleBased</i>
<i>black</i>	<i>ruleBased</i>
<i>black</i>	<i>alphaBeta</i>
<i>white</i>	<i>alphaBeta</i>

Marking:

- 1 mark for each element added to existing rows
- 2 marks for completing rows (ranges from 0-2, since it is a greedy algorithm)
- 1 for each missing pair (pair of values that is not covered)
- 0 if the algorithm was not followed, e.g., just listing tuples

Question 14: Metamorphic testing (4 points)

Erläutern Sie am Beispiel einer einfachen Schimpfworterkennung in einer Chatbot App (basierend auf dem Eingabetext) die Anwendung metamorphen Testens. Geben Sie die metamorphische Beziehung an, welche Sie voraussetzen.

Explain how metamorphic testing may be applied to profanity detection in a chat bot app (input is a text). Clearly state the metamorphic relation you assume to hold.

Marking:

- 2 marks the oracle, e.g., adding any filler text "Please answer briefly." should neither change the result from positive to negative.
- 2 marks for how to use it to compare two test outputs, e.g., feed original and extended text and compare results

Question 15: White-box Testing

- (a) (6 points) Zeichnen Sie ein Kontrollflussdiagramm für den angegebenen Code (die Zeilen sind so ausgerichtet, dass das Diagramm neben dem Code passen sollte). Beschriften Sie jeden Knoten mit der zugehörigen Zeilennummer.

Draw the control flow graph for the given code snippet (the lines are spaced to draw the graph next to the code). Label each node with the corresponding line number.

```
1 public double calc(double r, int n) {  
2     double x = 1;  
3     if (r > 1.8) {  
4         for (int i = 0; i < n; i++) {  
5             if (x > 3) {  
6                 x = 1;  
7             } else {  
8                 x = x * r;  
9             }  
10        }  
11    }  
12    return x;  
13 }
```

Marking:

- 1 for each wrong edge and each missing node (down to 0/6)

- (b) (3 points) Geben Sie eine minimale Testsuite für eine maximale Anweisungsabdeckung an. Provide a minimal set of test cases for maximal statement coverage.

Marking:

Example: (r=5, n=2, x=1); (-1 if not minimal)

0 in total, if these are not test cases, i.e., if the input is not clear. It is OK if only the input of each tuple is given. Variable names are optional, order of parameters alone is sufficient, i.e., (5, 2, 1) for the above suite.

one is sufficient.

- (c) (2 points) *Geben Sie eine minimale Testsuite für eine maximale Branch-Coverage an.*
Provide a minimal set of test cases for maximal branch coverage.

Marking:

Example: (r=5, n=2, x=1); (r=0, n=0, x=1), we cannot get better as the first if needs a false branch as well (-1 if not minimal)

- (d) (2 points) *Erklären Sie die Beziehung zwischen Bedingungsüberdeckung und MC/DC.*
Explain the relation between condition coverage and MC/DC.

Marking:

mc/dc (modified condition / decision coverage) subsumes condition coverage

- (e) (2 points) *Berechnen Sie die zyklomatische Komplexität des Programms.*
Compute the cyclomatic complexity of the program.

Marking:

based on control flow graph: $\#Edges - \#Nodes + 2 \cdot \#Components$

-1 if formula incorrect

-1 if computation incorrect

(computation must be based on control flow graph provided in (a))

Question 16: Agile manifesto (4 points)

Nennen Sie zwei der Grundsätze (in der Form „x über y“) des Agilen Manifests. Wählen Sie für jeden Grundsatz eine Praktik des Extreme Programming (XP) und benennen Sie kurz, wie diese die Werte abbildet.

List two of the values (in the form “x over y”) of the Agile manifesto. For each value, pick a practice of Extreme Programming (XP) and briefly explain how it reflects the value.

Marking:

1 mark x2 per value from below (must be of the same form, but wording can be different)

- Individuals and interactions over processes and tools
- Working software over comprehensive documentation
- Customer collaboration over contract negotiation
- Responding to change over following a plan

1 mark per example, e.g., from Small releases, Simple design, Test first, Refactoring, Pair programming, Continuous integration, On-site customer

Question 17: Code Smells (5 points)

Charakterisieren Sie Code Smells. Nennen Sie zwei Beispiele für Code Smells und erklären Sie, wie diese der Charakterisierung entsprechen.

Briefly characterize the concept of code smells. Give two examples and explain how these fit the characterization.

Marking:

1 mark for characterization: syntactically correct, but a structural problem that might cost in the long run.

1 mark for each example (L9, S 21-28).

1 mark for each explanation of how it relates to the definition